

Excel Shrink: An Open-Source Tool for Reducing File Bloat in Excel Spreadsheets

ANONYMOUS AUTHOR(S)

SUBMISSION ID: 1503

Excel files are widely used for data exchange across various domains. However, as spreadsheets grow in complexity, they often suffer from file bloat, leading to significant challenges in file size and performance management. We investigated Excel files from the German Federal Statistical Office (Destatis) and observed extremely long load times in Microsoft Excel and crashes due to out-of-memory exceptions in software packages used for reading Excel files. Our analysis revealed that these files contain format definitions invisible to the user, which occupy orders of magnitude more storage space than the actual data.

While some existing tools attempt to address this issue, they often fail to fully resolve the problem or introduce new issues, such as altering the visual appearance of spreadsheets. In this paper, we present a comprehensive analysis of the root causes of Excel file bloat, documenting issues that have been partially addressed by existing tools but remained undocumented, and uncovering additional issues not resolved by these tools. We also analyze why these tools impact the visual integrity of Excel files.

We developed an open-source tool, Excel Shrink, and its graphical user interface counterpart, Excel Shrink UI, which effectively address both the known and previously unresolved root causes of Excel file bloat without compromising the visual appearance of the spreadsheets. Excel Shrink UI enables users to optimize their Excel files without the need for programming knowledge or command-line tools. To evaluate our tools, we downloaded all Excel files from Destatis and optimized them using Excel Shrink. Our results show that our tools can reduce file sizes by up to 99.99% without affecting the visual integrity of the files.

CCS Concepts: • **Applied computing** → **Spreadsheets**; • **Information systems** → **Data cleaning**; • **Software and its engineering** → **Extensible Markup Language (XML)**; • **Human-centered computing** → **Graphical user interfaces**; • **Theory of computation** → **Data compression**.

Additional Key Words and Phrases: Excel File Optimization, XML Processing, Data Compression, Web Scraping, File Size Reduction, XLSX Format, Data Cleaning, Data Preprocessing, Open Data, Spreadsheet Efficiency, Data Management, Destatis

ACM Reference Format:

Anonymous Author(s). 2025. Excel Shrink: An Open-Source Tool for Reducing File Bloat in Excel Spreadsheets. In . ACM, New York, NY, USA, 23 pages. <https://doi.org/10.1145/XXXXXXX.XXXXXXX>

1 INTRODUCTION

Excel files are widely utilized as a format for data exchange across various domains, including finance, statistics, business, and research [22]. Their versatility lies in *Excel*'s capability to store tabular data, perform complex calculations, automate workflows using macros, and visually represent data through charts and graphs. [7, 23] This flexibility makes *Excel* indispensable in many organizations. However, as spreadsheets grow in complexity, significant challenges arise regarding file size and performance management, prompting novel solutions [5, 6, 27, 28].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SIGMOD '25, June 22–27, 2025, Berlin, Germany

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/25/06...\$15.00

<https://doi.org/10.1145/XXXXXXX.XXXXXXX>

Unlike simpler formats such as CSV, which transparently display stored data, *Excel* files use a more complex structure. *Excel* employs the *Office Open XML* format, introduced in *Excel 2007*, where workbook contents are stored as a collection of XML files within a compressed ZIP archive [12]. This XML-based architecture enables advanced functionality, such as separating data from its presentation layer. For instance, *Excel* centralizes repeated strings in a Shared String Table[9], thus reducing redundancy by storing each text entry once and referencing it elsewhere in the workbook. Furthermore, formatting details—such as fonts, cell styles, and colours—are stored separately from the data itself in dedicated XML files, allowing *Excel* to reconstruct the sheet’s visual appearance when it is opened.

1.1 *Excel*’s Internal Storage Mechanisms

To minimize file size, *Excel* makes extensive use of compression techniques. Once the workbook’s data, styles, and structure are defined in XML, these files are bundled into a ZIP archive. The ZIP compression algorithm is particularly effective in this context due to the repetitive nature of XML tags and structures. For example, in a large spreadsheet, tags like `<row>` and `<c>` (for cells) can appear hundreds of thousands of times. The compression algorithm identifies these repetitive patterns and encodes them with shorter representations, reducing the overall file size [11]. Without compression, the verbosity of XML would result in substantially larger files.

Despite achieving considerable space savings, this approach introduces significant inefficiencies. The separation between the visible content seen by the user and the underlying storage structure can lead to scenarios where redundant or unused data continues to occupy considerable amounts of space in the file. These inefficiencies are not immediately apparent to users but can drastically increase file sizes, reducing the overall performance of both *Excel* and the systems that interact with it [6, 27].

One common issue arises when users apply formatting to large ranges of cells that contain no actual data. For example, a user might highlight an entire row or column to change its font style or background colour, but only a subset of the cells are populated with data. The formatting persists across all selected cells, including those that remain empty, creating unnecessary XML records that *Excel* must store in its underlying files [4, 10].

Similarly, duplicating entire sheets within a workbook can carry over both data and formatting, even when only a portion of the original sheet is needed. Users often duplicate sheets to retain formulas, styles, or layouts, but this also transfers unnecessary metadata and formatting definitions from the original sheet, including empty rows, and hidden columns. These redundancies are copied wholesale, and in many cases, users do not realize that they are preserving and propagating unused content. Over time, as sheets are copied and modified, the workbook’s file size balloons, with *Excel* continuing to store and manage a growing list of redundant elements that serve no practical purpose [13].

These inefficiencies not only increase storage requirements but also degrade *Excel*’s performance. Larger files take longer to open, save, and modify [13, 24]. In enterprise environments, this can lead to significant delays when handling shared workbooks or automating data processing tasks through scripts and applications that interface with *Excel*. The excessive size of the files puts a strain on memory and processing resources, leading to a slower user experience and increased frustration in collaborative settings where files are frequently shared or integrated with other systems. These inefficiencies, therefore, represent a considerable challenge for users working with large datasets or managing multiple *Excel* files.

Existing tools and features, such as the *Excel Performance* tab, *NXPowerLite Desktop*, and the *Inquire* Add-in, aim to reduce *Excel* file sizes. However, these tools have notable limitations:

- **Closed Source and Lack of Transparency:** All existing tools are proprietary and do not disclose the underlying methods they use to address file bloat. This opacity prevents users and researchers from understanding the root causes of the problem and limits the potential for further optimization.
- **Incomplete Solutions:** These tools often fail to fully resolve the issue, neglecting certain types of redundant content such as unnecessary column definitions. This means that significant portions of file bloat remain unaddressed.
- **Alteration of Visual Appearance:** In some cases, particularly with the *Excel Performance* tab, these tools significantly alter the visual appearance of spreadsheets. For users who need to preserve the original formatting, such changes are unacceptable.

Our research was motivated by the need to understand and address these shortcomings. We set out to identify the specific causes of file bloat in *Excel* files and to develop a solution that overcomes the limitations of existing tools. This involved:

- **Analyzing the Root Causes of File Bloat:** We performed a systematic investigation into why *Excel* files become excessively large, focusing on hidden inefficiencies such as redundant column definitions and unused cell formats that have not been previously documented in the scientific literature.
- **Evaluating Existing Tools:** We examined the limitations of existing solutions, including analyzing the damage they can cause to spreadsheets' visual layouts by inspecting the altered XML structures after processing.
- **Developing a Superior Solution:** We created *Excel Shrink*, an open-source tool that effectively reduces file sizes by addressing all identified causes of file bloat without altering the spreadsheets' appearance.

Our contributions can be summarized as follows:

- We provide a comprehensive analysis of the root causes of file bloat in *Destatis Excel* files, uncovering previously undocumented issues such as unnecessary column definitions.
- We reveal the limitations of existing proprietary tools, explaining why they fail to fully address file bloat and how they can unintentionally alter the visual appearance of spreadsheets.
- We develop *Excel Shrink*, an open-source tool that surpasses existing solutions by effectively reducing file sizes while preserving the visual formatting of *Excel* files. For users who prefer a user-friendly interface without the need to install *Python* or use command-line tools, we developed *Excel Shrink UI* [3].
- We evaluate *Excel Shrink* on a comprehensive dataset of *Destatis Excel* files. Our results demonstrate that *Excel Shrink* can reduce file sizes by up to 99.99% without compromising the usability or appearance of the *Excel* files

By addressing the previously unexplored problem of hidden redundancies in *Excel* files and providing a transparent, open-source solution, our work fills a significant gap in the field of spreadsheet optimization.

2 EXCEL SHRINK

The objective of *Excel Shrink* is to reduce the size of *Excel* files by removing content that is invisible to the user and likely unintentionally included in the file. This includes empty cells, cells containing only whitespace, cells with non-visible formatting, and unused rows and columns.

2.1 Deletion of White Spaces

Excel users lack a straightforward method to identify which cells in a worksheet contain only whitespace characters. To highlight these whitespaces, we have marked cells containing exclusively whitespaces in red within this worksheet, which includes 4,121 such instances (see Figure 1) [18].

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
843	17	=	Private Konsumausgaben	1773.842												
844																
845			Jahr: 2022													
846																
847	1	Land- und Forstwirtschaft, Fischerei	8.644	5.249	0.646		0.928	0.096					1.725			
848	2	Bergbau, Verarbeitendes Gewerbe und														
849	3	Baugewerbe	38.616	17.324	0.285	1.489	7.994	4.182	0.671	1.974	0.079	2.106		1.352	1.180	
850	3	Energieversorgung	72.984				72.984									
851	4	Kraftfahrzeughandel, Instandhaltung und														
852		Reparatur von Kraftfahrzeugen	117.681	0.055	0.030	0.044	0.021	0.029	0.004	114.264		3.128		0.102	0.004	
853	5	Großhandel (ohne Handel mit Kraftfahrzeugen)	43.948	2.306	14.533	2.275	10.421	6.312	1.292	0.836	0.312	4.410			1.251	
854	6	#####	667.781	189.385	43.070	71.811	12.701	106.532	38.456	80.737	6.321	84.665		0.569	33.534	
855	7	Verkehr und Lagerei	48.434							44.614	3.634	0.186				
856	8	Gastgewerbe	107.513	1.287	0.559	0.544				0.844	0.492	0.884		102.355	0.548	
857	9	Information und Kommunikation	43.568								28.444	13.780		0.180	1.164	
858	10	Finanz- und Versicherungsdienstleister	98.956												98.956	
859	11	Grundstücks- u. Wohnungswesen, Öffentliche														
860		Dienstleister, Erziehung, Gesundheit, sonstige														
861		Dienstleister	630.654	0.012	1.311	1.187	352.877	10.997	54.970	17.303	0.018	83.327	15.358	1.835	91.459	
862	12	= Konsumausgaben der priv. Haushalte im Inland	1878.779	215.618	60.414	77.350	457.926	128.148	95.393	260.572	39.300	194.211	15.358	106.393	228.096	
863	13	+ Konsumausgaben der inländer in der übr. Welt	71.103													
864	14	= #####	25.623													
865	15	= Konsumausgaben der inländ. priv. Haushalte	1924.259													
866	16	+ Konsumausgaben der priv. Organisationen o.E.	58.421													
867	17	= Private Konsumausgaben	1982.680													
868																
869																
870																
871																
872																
873																
874																

Fig. 1. Cells containing only whitespaces are highlighted in red.

Listing 1. Cell D844 in sheet28.xml

```
<row r="844" spans="1:16" ht="14.1" customHeight="1">
  <c r="D844" s="428" t="s">
    <v>102</v>
  </c>
```

To detect these whitespaces, it is insufficient to examine the cell itself, as even a single space is stored by *Excel* in the file for recurring texts, even if the text appears only once in the entire *Excel* file. Cell D844 appears in the XML as shown in Listing 1.

The row is marked by the `<row>` tag with the attribute `r="844"`, indicating that it is the 844th row. Within the row, the `<c>` tag represents a cell, identified by the attribute `r="D844"`. The attribute `t="s"` specifies the cell type; in this case, `s` stands for *String* [14].

The cell's value is stored within the `<v>` tag, where the value `102` is specified. This value is interpreted as an index in a string table, referencing a specific text in a separate list defined in the *Excel* file, namely the `sharedStrings.xml` file.

Since *Excel* stores even single spaces in the Shared String Table, merely inspecting the cell's content is insufficient to detect whitespaces. By examining the shared strings, we can identify entries that contain only whitespace characters or are empty, allowing us to determine which cells can be safely deleted. To locate the value, we must search for the `<sst>` node in the `sharedStrings.xml` file, as shown in Listing 2.

Listing 2. Shared String Table (sst) in sharedStrings.xml

```

197 <sst count="17962" uniqueCount="732">
198   <si>
199     <t>062</t>
200   </si>

```

The provided XML excerpt displays a portion of the Shared String Table (SST) in an *Excel* file [9]. The Shared String Table is a structure that allows for the storage and management of shared strings within an *Excel* workbook. Instead of storing each string separately in every cell, each unique string is stored once in the Shared String Table. This approach saves storage space, as cells then reference these strings via indices. Each entry in this table has an ascending index number, starting at 0 for the first string, 1 for the second, and so forth. The count attribute indicates the total number of strings in the table, while uniqueCount represents the number of unique strings. In this case, there are 17,962 strings in total, of which 732 are unique. This means that many strings in the table are used multiple times.

Within the SST, there are elements of type <si> (Shared Item), each containing a string. In this example, the <t> element displays the actual text of the string stored in the table. Here, the string 062 is represented, which can be referenced by an index in a cell.

To find the string corresponding to index 102, we must look for the 103rd entry (since indexing starts at 0), which is shown in Listing 3.

Listing 3. Empty string at index 102 in sst

```

217 <si>
218   <t xml:space="preserve"></t>
219 </si>

```

Within the <si> element is a <t> element containing the actual text of the string. In this specific case, the content of the <t> element is empty, indicating that no text is stored. The attribute xml:space="preserve" suggests that this represents a whitespace character. Therefore, all cells referencing index 102 for the reusable string can be deleted, as they contain no visible content.

2.2 Deletion of Empty Columns

The first indication of excessive column definitions can be observed in the horizontal scrollbar when opening the worksheet, as shown in Figure 2 [21, Sheet: 0902_T_NW]. If the entire data range is already within the visible area and yet the scrollbar thumb is very narrow and small compared to the scrollable area, it indicates that a large number of column definitions are making the worksheet significantly wider than the actual data range.

Scrolling into this area, the user may notice that among columns with uniform widths, some columns occasionally have differing widths, as shown in Figure 3.

These column definitions are specified in *Excel* within the file sheetX.xml under the cols node. In our example, the definition of the initial columns is shown in Listing 4.

In this XML excerpt, the <cols> element represents the definition of multiple columns in an *Excel* worksheet. Each <col> declaration contains specific information about one or more columns. The min and max attributes specify the range of columns that the <col> element covers. For example, min="1" max="1" pertains only to column 1, while min="5" max="9" describes columns 5 through 9. The width attribute specifies the width of the columns in *Excel* units. The customWidth="1" attribute indicates that the column width is user-defined, meaning it has been manually adjusted rather than using the default value.

The first five <col> definitions relate to the data range. However, the last definition, min="10" max="18", pertains to a redundant range. This section defines columns that lie outside the actual

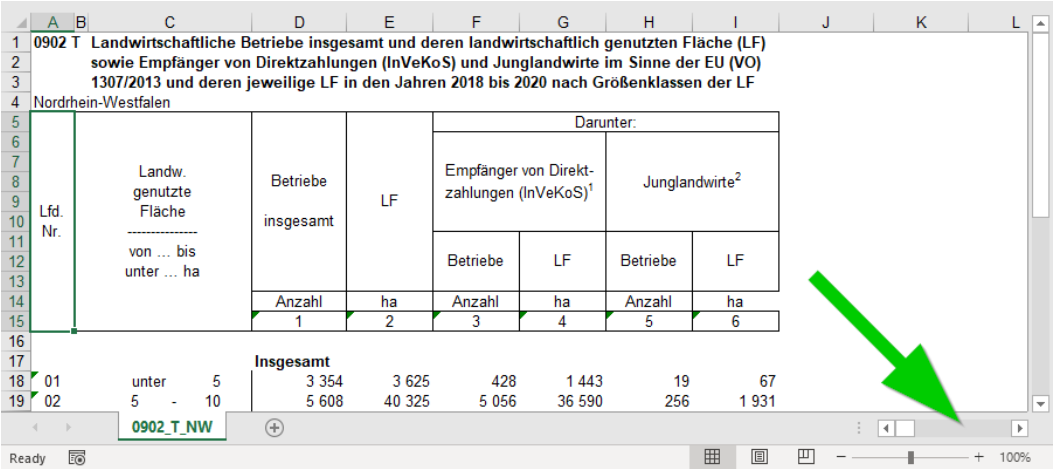


Fig. 2. Very wide scrollable area and very narrow scrollbar thumb

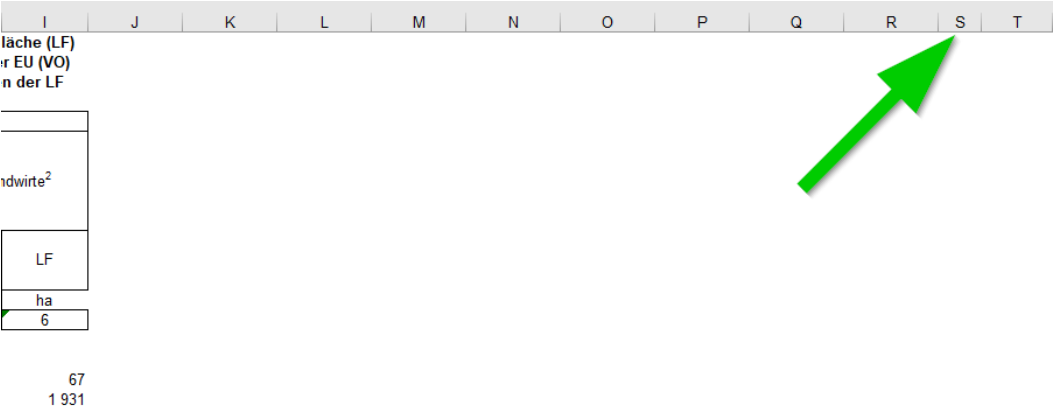


Fig. 3. Custom Column Width for a Column Outside the Data Range

data range in the worksheet, specifically columns starting from column J, thereby representing unnecessary or unused definitions.

Listing 4. Initial column definitions

```
<cols>
  <col min="1" max="1" width="5.21875" style="39"
    customWidth="1" />
  <col min="2" max="2" width="1.77734375" style="39"
    customWidth="1" />
  <col min="3" max="3" width="20" style="39" customWidth="1" />
  <col min="4" max="4" width="11.5546875" style="39"
    customWidth="1" />
```

```
<col min="5" max="9" width="10.5546875" style="39"
  customWidth="1" />
<col min="10" max="18" width="11.5546875" style="54"
  customWidth="1" />
```

During our investigations, we could not determine how these column definitions originated. We hypothesize that the worksheet was duplicated, and the original worksheet contained this number of columns with the corresponding widths. However, this hypothesis is contradicted by the fact that the number of columns in the files we examined approaches *Excel*'s maximum limit of 16,384 columns. Figure 4 illustrates such a case [21, Sheet: 0902_T_NW]. This worksheet contains 2,403 column definitions, and after excluding the five actually used column definitions, 2,398 redundant definitions remain.

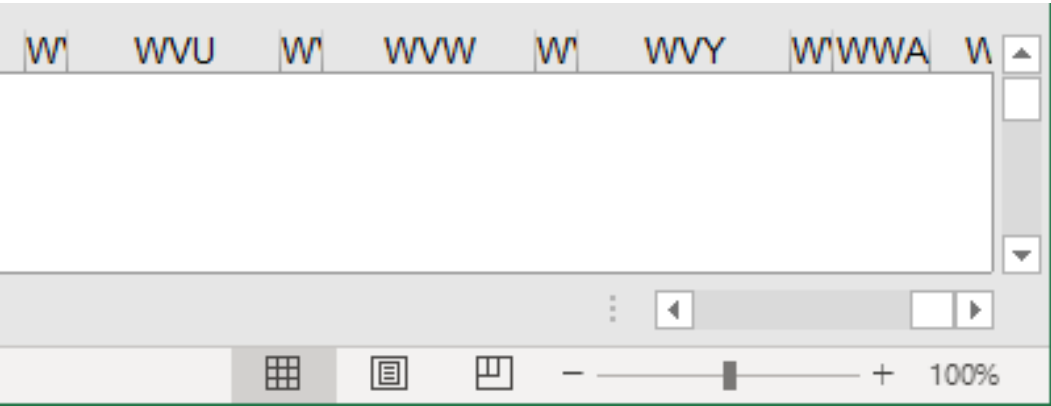


Fig. 4. Excessive Column Definitions in a Worksheet

Columns WWA and the partially visible WWB correspond to the 16,147th and 16,148th columns, respectively. It seems improbable that an original worksheet would require the utilization of such an extensive number of columns. Further research is required to determine how such patterns emerge in *Excel* files.

The last visible columns in Figure 4 are shown in Listing 5. The first column `min="16146" max="16146"` has a fixed width of 2.21875 and refers to column WWZ, which is visible as the column immediately to the left of WWA but is too narrow to display more than the letter W. The second column `min="16147" max="16147"` defines column WWA with a width of 5.21875. The final definition `min="16148" max="16384"` covers a larger range from column 16148 to 16384, corresponding to columns WWB through XFD (the last column in *Excel*). This explains why scrolling all the way to the right via the scrollbar lands on column WWB. The penultimate definition specifies where the data range ends, thereby also defining where the scrollable range ends, and the final definition simply assigns a default width to all remaining columns. Surprisingly, these column definitions extend all the way to *Excel*'s maximum limit of 16,384 columns. We have no explanation for why these user-defined column widths appear up to just before the maximum allowable column, but not exactly at it.

To delete these columns, the following steps must be taken:

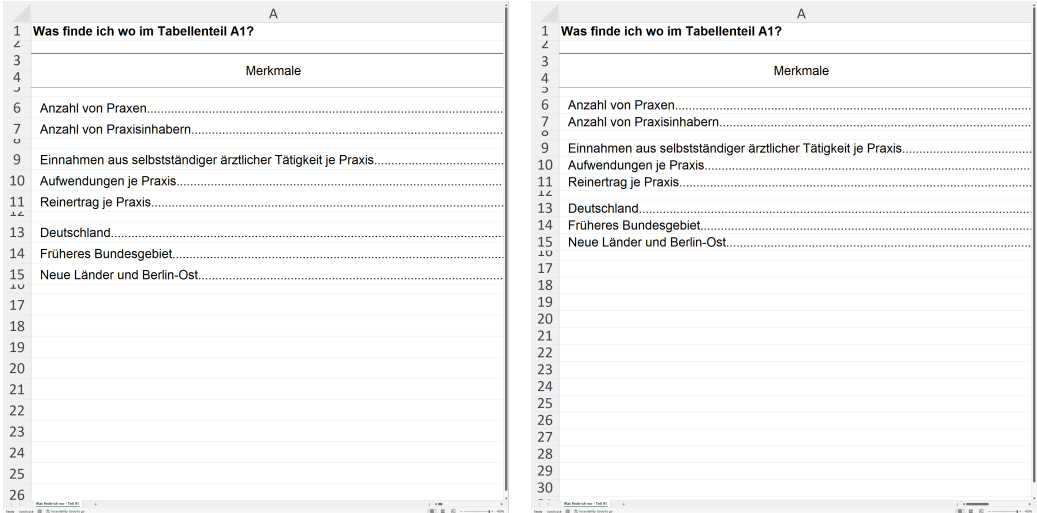
- (1) Identify the actual right boundary of the data range and cell formats.
- (2) Determine the first redundant column definition and save the `min` value to use it later.

- (3) Delete all redundant column definitions except for the last one.
- (4) Overwrite the min value of the last column definition with the saved value of the first redundant column definition.

Listing 5. Final column definitions in the worksheet

```
<col min="16146" max="16146" width="2.21875" style="39"
  customWidth="1" />
<col min="16147" max="16147" width="5.21875" style="39"
  customWidth="1" />
<col min="16148" max="16384" width="8.77734375" style="39"
  customWidth="1" />
</cols>
```

Following these steps is crucial. We experimented with merely deleting the excess columns without ensuring that the last column definition extended to the final possible column (16,384), but this caused *Excel* to mark these worksheets as corrupted and perform certain repairs. Part of these changes included altering the default row height, resulting in some worksheets appearing visually altered, as shown in Figures 5a and 5b. On the left is the original, and on the right is the repaired version by *Excel*. For instance, row 6 has changed in height due to the repair, causing visible shifts in the worksheet. Correctly deleting column definitions while ensuring the last column definition extends to the column limit of 16,384 prevents this error.



(a) Original worksheet before repair (b) Worksheet after repair by *Excel*

Fig. 5. Visual differences due to changes in default row height

2.2.1 Hidden Columns. Removing columns is not recommended if there are hidden columns. In spreadsheets like *Excel*, cells define only where text begins. If the text is longer than the cell, it typically extends over adjacent cells unless those cells also contain text. Some worksheets rely on the assumption that text will overflow into adjacent cells, causing the range of actually used cells to span multiple columns. In such cases, it's important not to delete columns outside the range of cells

with values. Doing so could shift hidden columns to the next position, causing text from previous cells to overflow into them. This is depicted in Figures 6a and 6b. Since the texts are only defined in columns A and B, all cells from column C onwards are outside the range of cells with values and are thus considered for deletion. However, column C is already hidden, causing the texts to be cut off.

When *Excel Shrink* detects hidden columns, it avoids removing them to ensure that the worksheet’s visual representation is not altered.

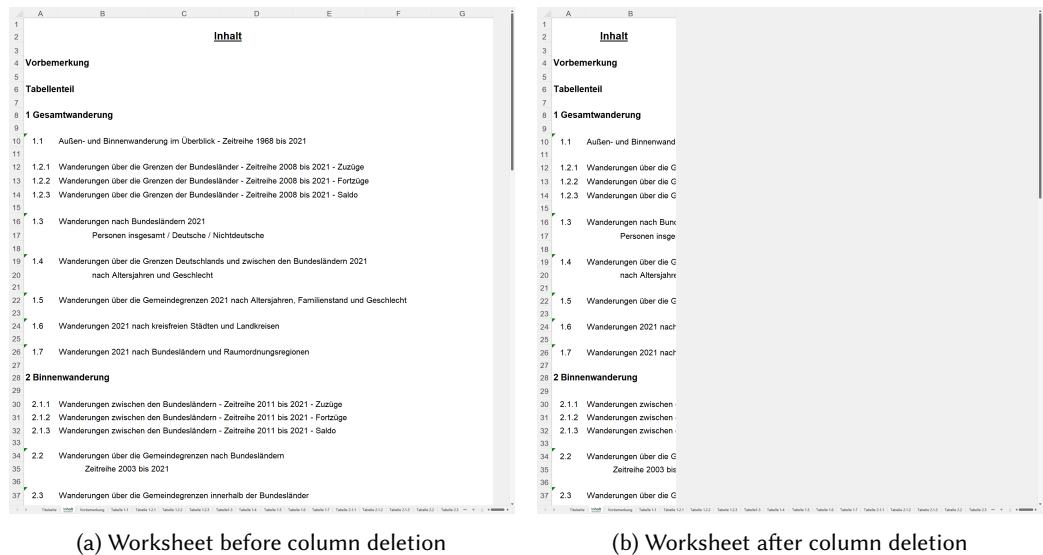


Fig. 6. Effect of deleting columns when hidden columns are present

2.3 Deletion of Empty Cells

Some of the worksheets we examined contained vast numbers of cells without any content. The most extreme case was a worksheet with 8,141 cells containing values and 5,237,075 cells without values, meaning that only 0.15% of the stored cells actually had a value [19, Sheet: csv-71717-25]. Listing 6 shows the last row of this worksheet.

Listing 6. The last row in the worksheet csv-71717-25

```
<row r="1048576" spans="3:7" ht="12.75" customHeight="1">
  <c r="C1048576" s="538"/>
  <c r="D1048576" s="537"/>
  <c r="E1048576" s="537"/>
  <c r="F1048576" s="537"/>
  <c r="G1048576" s="541"/>
</row>
</sheetData>
```

Unlike the whitespaces, these cells now contain no value and therefore do not need to be cross-referenced with the sharedStrings file. However, it is possible that such empty cells may have a background color assigned, as is the case with some cover sheets used for page design or for highlighting table headers. Additionally, these cells might have borders applied.

To determine this, one must examine the `s` attribute, which stands for the style index applied to the cell. The values `s="538"`, `s="537"`, and `s="541"` correspond to indices in a lookup table where all cell styles are stored. The styles matching these style indices can be found in the `xl/styles.xml` file. The relevant excerpts are shown in Listing 7.

Listing 7. Cell styles in `styles.xml` for `csv-71717-25`

```
<cellXfs count="778">
  <!--538-->
  <xf numFmtId="0" fontId="0" fillId="0" borderId="0" xfId="0"/>
  <!--539-->
  <xf numFmtId="0" fontId="4" fillId="0" borderId="0" xfId="3"
    applyNumberFormat="1" applyFont="1"/>
  <!--542-->
  <xf numFmtId="0" fontId="1" fillId="0" borderId="0" xfId="1"
    applyNumberFormat="1" applyFont="1" applyAlignment="1">
    <alignment horizontal="right"/>
  </xf>
```

The XML structure within `<cellXfs>` contains formatting properties defined for cells in an *Excel* file. Each `<xf>` element represents a formatting rule. Similar to the Shared Strings file, formatting rules are indexed based on their order in the list, starting with the first at 0, the second at 1, and so on. For the IDs `s="538"`, `s="537"`, and `s="541"`, one must look for the 539th, 538th, and 542nd elements in the list, respectively. The attributes control the cell's appearance. The `numFmtId` attribute specifies the number format used for the cell, while `fontId` describes the font ID. `fillId` refers to the fill color, and `borderId` defines the border properties. The `xfId` references a format template from which the cell inherits properties. The attributes `applyNumberFormat`, `applyFont`, and `applyAlignment` indicate whether the corresponding number format, font, and text alignment are applied to the cell. If the `alignment` element is present, it specifies the text alignment within the cell, such as right alignment. This structure defines the formatting rules used for the display and layout of the cell contents.

Inspection of these styles reveals that these cells only apply styles that are not visible for empty cells. They use different fonts, and cell G1048576 is additionally right-aligned. Therefore, all these cells are redundant and can be deleted accordingly.

Another case arises when the formatting rules include borders. If such cells are simply deleted, as occurred in previous versions of our script, issues arise, as seen in Figure 7 [16, Sheet: 2_(1)]. In Figure 7a, all borders around the cells in the table header are visible. In Figure 7b, only the borders adjacent to cells with content remain visible. For example, the border between cells A2 and A3 is missing.

Cell A2 appears in the XML as shown in Listing 8.

Listing 8. Cell A2 in `sheet7.xml`

```
<cell r="A2" s="577"/>
```

Here, the style ID is `s="577"`. It defines the underlying cell formatting, which in turn includes the ID for the border definition.

In the `styles.xml` file under the `<cellXfs>` node, we search for the `<xf>` node with ID 577, which corresponds to the 578th `<xf>` node. This is shown in Listing 9.

Listing 9. Style with implicit ID 577 in `styles.xml`

	A	B	C	D	E	F	G	H	I	J	K	L
1	2 Straßenverkehrsunfälle, Verunglückte, Bestand an motorisierten Fahrzeugen und Bevölkerung 1906 – 2021											
2		Polizeilich erfasste Unfälle ¹			Verunglückte ^{1 2}			Bestand an motorisierten Fahrzeugen ³			Bevölke- rung ⁷	
3		insgesamt	davon mit		insgesamt	davon		insgesamt	darunter			
4			Personen- schaden	Sach- schaden		Getötete	Verletzte		Motor- räder mit amt. Kenn- zeichen ⁴	Personen- kraftwagen ⁵	Güterkraft- fahrzeuge ⁶	
5												
6												
7												
8												
9		Anzahl										1 000
11		Reichsgebiet ⁸										
13	1906	2 290	1 303	987	1 570	51	1 519	.	.	.	.	.
14	1907	4 864	2 230	2 634	2 564	145	2 419	27 026	15 954	10 115	957	.
15	1908	5 069	2 342	2 727	2 771	141	2 630	36 022	19 808	14 671	1 543	.

(a) Original version with correct cell borders and spacing

	A	B	C	D	E	F	G	H	I	J	K	L
1	2 Straßenverkehrsunfälle, Verunglückte, Bestand an motorisierten Fahrzeugen und Bevölkerung 1906 – 2021											
3		Polizeilich erfasste Unfälle ¹			Verunglückte ^{1 2}			Bestand an motorisierten Fahrzeugen ³				Bevölke- rung ⁷
4		insgesamt	davon mit		insgesamt	davon		insgesamt	darunter			
5			Personen- schaden	Sach- schaden		Getötete	Verletzte		Motor- räder mit amt. Kenn- zeichen ⁴	Personen- kraftwagen ⁵	Güterkraft- fahrzeuge ⁶	
6												
7												
8												
9		Anzahl										1 000
11		Reichsgebiet ⁸										
13	1906	2 290	1 303	987	1 570	51	1 519					
14	1907	4 864	2 230	2 634	2 564	145	2 419	27 026	15 954	10 115	957	
15	1908	5 069	2 342	2 727	2 771	141	2 630	36 022	19 808	14 671	1 543	

(b) Corrupted version with removed borders and spacing

Fig. 7. Erroneous removal of cells with borders and deletion of rows with custom heights

```
<xf numFmtId="0" fontId="8" fillId="2" borderId="5" xfId="0"
  applyFont="1" applyFill="1" applyBorder="1"
  applyAlignment="1"/>
```

This `<xf>` node has the `borderId` attribute set to index 5. Similar to cell styles, this index indicates that the 6th `<border>` node within the same `styles.xml` file is referenced.

Listing 10 shows the 6th `<border>` node with implicit ID 5.

Listing 10. Border node with implicit ID 5 in `styles.xml`

```
<borders count="12">
  <!-- ... -->
  <border>
    <left/>
    <right/>
    <top/>
    <bottom style="thin">
      <color indexed="64"/>
    </bottom>
    <diagonal/>
  </border>
```

Each `<border>` node contains the child nodes `left`, `right`, `top`, `bottom`, and `diagonal`. If these nodes lack attributes or child nodes, it means that the cell has no border. In this case, the `bottom` node has the attribute `style` set to "thin" and includes the `color` node with the attribute `indexed="64"`, indicating that the cell has a thin black line at the bottom. Cells identified as having borders must not be deleted. Only cells that have neither content nor borders (i.e., their border nodes lack attributes indicating border presence) may be safely deleted.

2.4 Inverted Fill

During the development of *Excel Shrink*, we made an interesting discovery. We found cells that had no content and no border but a fill definition that specified no fill should be applied. As a result, these cell definitions were deleted, and the cells turned red. We discovered that the entire row containing the cell had a red fill applied, and each cell within that row had a definition specifying that no fill should be applied to override the red fill color. As a user, one does not have any means to identify this by looking at the sheet. All the visible cells had the overridden fill color, and the rest of the cells were hidden behind hidden columns. Only after unhiding those columns did the issue become evident. Situations like these make preserving the visual appearance especially difficult.

We cannot simply decide to delete all cells that have a style applied that tells the cell to apply no fill because such a scenario might be useful in some situations. For example, if the whole row or column is intended to have a certain style and only some cells should apply no style as an exception to the rule. At the same time, in some scenarios, this might not be intended. The file becomes exceptionally large because some rows inadvertently have a style applied, and all cells in those rows up to the limit of possible columns define that this style should be overridden by no style.

To reduce the file size while preventing such changes to the display, we proceed as follows. First, we determine the style of the column and the row in which the cell is located. If either the column or row has a background color defined, we check whether the cell defines the same background color. In this case, we delete the cell. If the background color information of the cell differs from that of the column or row, we retain the cell. The same applies analogously to borders. If the column or row defines a border, we check whether the cell defines the same border. In this case, we delete the cell. If the border information of the cell differs from that of the column or row, we retain the cell.

3 DELETION OF EMPTY ROWS

Similar to cells without content but with visible formatting, rows may contain no defined cells but still have a defined height. Such rows must not be deleted because their height can affect the worksheet's layout. If such a row is deleted, an issue occurs as depicted in Figure 7b compared to the original in Figure 7a. The spacing between the header and the table shrinks because the intermediate row, which served as spacing between the header and the table, is deleted, and the default row height for the worksheet is applied, which is smaller than the custom height of the row. This row is shown in Listing 11.

Listing 11. Row with custom height

```
<row r="2" spans="1:12" ht="8.25" customHeight="1">
```

The attribute `customHeight` means that this row has been assigned a user-defined height, specified by the `ht` attribute: 8.25. Therefore, this row must not be deleted. However, during our examination of the worksheets, we found many that contained an immense number of empty rows, which had defined heights.

In the most extreme case in our analysis, a worksheet contained 1,048,576 defined rows. Of these, only 94 rows had content, which corresponds to approximately 0.00896% of the total rows [17,

Sheet: Tabelle 1.2.1]. The remaining 1,048,482 rows had a defined height but were outside the data range.

Listing 12. The last row in the worksheet

```
<row r="1048576" ht="12.75" hidden="1" customHeight="1"/>
```

After the 96th row, which is the last one with content, there are 1,048,482 rows that are identical to the last row shown in Listing 12, except for the `r` attribute. These rows all have the same height and are hidden, as indicated by the `hidden="1"` attribute. Although they are not displayed to the user, they have a user-defined height of 12.75 units, which is why they are still stored in the file. Row 1,048,576 corresponds to the last row, which is the maximum number of rows allowed in *Excel*. If *Excel* allowed a higher number of rows, the wasted file storage would be significantly greater.

We cannot rely on a row simply not having a defined height to decide whether it can be deleted. However, we also cannot simply delete rows that have a user-defined height, as they may be important for the worksheet's layout. Therefore, we must find another method to identify empty rows that can be safely deleted.

To ensure that deleting empty rows does not alter the worksheet's visual representation, we adopt a strategy similar to that used for removing columns. First, we determine the actual range where values, background colors, or borders are defined. Then, we iterate through all empty rows and delete them only if they lie outside this range. This approach allows us to significantly reduce the file size of some worksheets without altering their content or appearance.

3.1 Deletion of Redundant Print Areas and Print Titles

Excel allows users to define the area of a worksheet that should be printed on paper. This area can be manually defined or automatically set by *Excel*. In both cases, this area is stored in the `workbook.xml` file under the `<definedNames>` node [8]. Typically, this area is labeled as `_xlnm.Print_Area` and `_xlnm.Print_Titles`. However, these areas and titles are often also stored under the names `Print_Area` and `Print_Titles`. When *Excel* opens the file, it renames the entries `Print_Area` and `Print_Titles` to `_xlnm.Print_Area` and `_xlnm.Print_Titles`, respectively. This process creates a problem when there is already a *Defined Name* for the same worksheet with both `Print_Area` and `_xlnm.Print_Area`, as seen in Listing 13[17, `x1/workbook.xml`].

Listing 13. Redundant Print Area entries

```
<definedName name="_xlnm.Print_Area" localSheetId="12">'Tabelle
  2.1.1'!$A$1:$N$64748</definedName>
<!-- ... -->
<definedName name="Print_Area" localSheetId="12">'Tabelle
  2.1.1'!$A$1:$N$93</definedName>
```

The same applies to `Print_Titles` and `_xlnm.Print_Titles`. In this case, the entries cannot be renamed because it would lead to redundant entries. Instead, *Excel* presents a dialog to the user upon opening the file, asking the user to select a new name for the *Defined Name*.

If the first entry is renamed and the user clicks OK, the dialog reappears for the next entry. This process repeats until all redundant entries for both `Print_Area` and `Print_Titles` are renamed. However, the user cannot simply enter the same name repeatedly to shorten this process, as *Excel* will prevent the same name from being used twice for different *Defined Names* on the same worksheet.

In the files we examined, there were up to 60 redundant `Print_Area` and 55 `Print_Titles` entries. Renaming these entries offers no advantage to the user, as these entries are only stored in

the *Defined Names* accessible via *Excel*'s Formulas → Name Manager. Users might think that they are repairing the *Print Areas* or *Print Titles* by entering those new names, but this is not the case because those *Print Areas* and *Print Titles* are already migrated correctly. In reality, the user only renames redundant *Defined Names*.

In earlier versions of our script, we did not remove redundant entries from the files. This led to the problem that we could not perform our analysis on how long the *Excel* files took to open before and after reduction. The *Visual Basic for Applications (VBA)* script we used to open and close the files stopped when encountering files with redundant entries. To simplify the analysis and spare users the need to manually rename redundant entries, *Excel Shrink* was extended to automatically remove redundant entries.

3.2 Reporting Reduction Results

To provide a breakdown of how many bytes were saved by deleting whitespaces, empty cells, columns, and rows, we compare the file size before and after each reduction step. For this purpose, we calculate the size of the worksheet by converting the XML structure into a string, as it would be stored in the file, and compute the length of this string. We then perform a reduction step and again calculate the length of the new string to determine the difference caused by the reduction. Finally, we calculate the percentage reduction relative to the original size. These results are saved in a CSV table.

3.2.1 Determining the Sheet Name. To provide the user with a breakdown of the reduction per worksheet, it is necessary to determine the worksheet titles. However, through the worksheet files, we only know an ID indicated in the filename, such as *sheet1.xml*, *sheet2.xml*, and so forth. Unfortunately, we cannot determine the sheet title directly from the sheet file because the title is only associated with the individual sheets in the *xl/workbook.xml* file. This association is shown in Listing 14 [21]. The *sheetId* attribute should be ignored; the ID that maps the sheet title to the corresponding *sheetX.xml* is the *r:id* attribute. Since each value in this attribute starts with *rId*, the ID must be extracted by removing the first three characters. These IDs then provide a unique mapping to the worksheet titles, which can be used for output.

Listing 14. *workbook.xml* mapping IDs to worksheet titles

```
<sheets>
  <sheet name="Titelseite" sheetId="10" r:id="rId1"/>
  <sheet name="Inhalt" sheetId="11" r:id="rId2"/>
  <sheet name="Erläuterungen" sheetId="3" r:id="rId3"/>
  <sheet name="Allgemeines" sheetId="4" r:id="rId4"/>
  <sheet name="0901_T_D" sheetId="12" r:id="rId5"/>
  <sheet name="0901_T_BW" sheetId="13" r:id="rId6"/>
```

4 COMPARISON OF EXISTING TOOLS

We have identified three tools that also reduce the size of *Excel* files. One of these tools is a commercial program with a trial version available, named *NXPowerLite Desktop* [?]. Another tool is an *Add-in* for *Excel*, called *Excel Inquire*. Additionally, in the 2022 version of *Excel* for Web [25] and the 2024 version of *Excel* for Windows [26], *Microsoft* introduced the *Check Performance* feature, which allows for the reduction of *Excel* file sizes. We have tested these programs and compared sample results with our tool.

4.1 NXPowerLite Desktop

The program *NXPowerLite Desktop* is commercial software [?]. A 14-day trial version is available. Although the primary purpose of the program is to reduce the file size of *Excel* files by compressing embedded images, it also removes some, but not all, empty cells. However, it ignores all cells containing only whitespaces, redundant column definitions, and unnecessary rows. The program has limited batch functionality, allowing the selection and reduction of multiple files simultaneously. However, this is restricted to a maximum of 20 files.

4.2 Inquire Add-in

Inquire is an *Add-in* for *Excel* [15], available exclusively for *Excel* versions within *Office Professional Plus* and *Microsoft 365 Apps for Enterprise*. Other versions, such as *Office Home*, *Microsoft 365 Family* or *Single*, and *Microsoft 365 Business*, do not support this *Add-in*.

Compared to our solution, *Inquire* ignores whitespace cells but successfully deletes empty cell, column, and row definitions. However, in certain instances, a substantial number of these elements remain undeleted. For instance, in a worksheet used as a cover sheet, only 77 rows were utilized, and all rows from row 78 onwards were hidden. However, in the XML file, rows 78 to 65533 were defined along with cells from columns A to F, as demonstrated in Listing 15.

Listing 15. Row 65533 in sheetX.xml

```
<row r="65533" spans="1:7" ht="0" hidden="1" customHeight="1">
  <c r="A65533" s="68" />
  <c r="B65533" s="68" />
  <c r="C65533" s="68" />
  <c r="D65533" s="68" />
  <c r="E65533" s="68" />
  <c r="F65533" s="68" />
  <c r="G65533" s="68" />
</row>
</sheetData>
```

When applying the *Inquire Add-in* to such worksheets, they were not cleaned. A dialog indicated that because the default row height is 0, it is impossible to clean the sheet, and hence the sheet will be skipped.

Additionally, *Inquire* lacks batch processing capabilities. As a result, multiple files cannot be processed simultaneously. Instead, each file must be opened individually. Given that large files, which are typically the ones requiring size reduction, take at least one minute to open, this process becomes quite time-consuming. Opening several large files and subsequently using the *Inquire Add-in* to reduce their size involves a significant amount of time.

4.3 Excel Check Performance

Since 2022, *Excel* for Web has included a new feature named *Check Performance*, and since 2023, this feature has also been available in *Excel* for Windows. It can be accessed through the Review tab in the Ribbon menu. In our comparison with alternative tools, this function achieved the largest reduction in file size; however, it failed to remove unneeded column definitions and cells with whitespaces.

Check Performance was capable of outperforming *Excel Shrink* in specific cases without altering the worksheet's appearance. Nevertheless, it modified the worksheet in a manner that required

users to perform multiple steps to revert the changes made by *Check Performance*. This issue arose when a worksheet contained an actual value in a hidden row, and between this row and the next visible row, there were thousands of additional hidden rows without any values. For example, we had a worksheet whose visible content extended only to row 94. After making the invisible rows visible to verify our observations from the XML file of the sheet, we found that in row 65536, column A, the number 3 was stored, presumably by accident [17, Sheet: Tabelle 2.1.1].

In such scenarios, *Excel Shrink* is unable to remove the hidden rows because our strategy involves removing only content outside the range of actual values and visible formatting. Since the last row within the user-designated area contains an actual value, all intervening rows must be preserved. In contrast, *Check Performance* removes all these hidden rows. This action alone would typically result in the previously hidden and now deleted rows becoming visible again. To prevent this, *Check Performance* employs a workaround by setting the row height to zero for the entire worksheet. This creates the illusion that the rows are deleted when, in fact, they are merely not visible due to having no height.

However, this approach presents a significant drawback for users. Once users select all rows and attempt to make them visible again, the previously hidden rows remain invisible. To actually make all rows visible, users must select all rows in the worksheet and assign a new height. This method is disadvantageous because it also alters the height of rows that users may have previously customized. To preserve these custom height settings, users must follow a specific procedure: first, select all rows using the *Select All* button, then hold down the *Control* key and deselect all visible rows except the last one. Consequently, only the last visible row and all hidden rows remain selected. Users can then adjust the height of all previously invisible rows using the resize handle of the last visible row, thereby making them visible again.

Given that users may be surprised when attempting to make the rows visible in the usual manner but find that the rows remain hidden, we decided not to adopt the same strategy as *Check Performance*. Considering that the steps required to actually make these rows visible involve a learning curve and can be time-consuming—especially when the worksheet already contains several visible cells requiring multiple scrolls to deselect the hidden rows—we opted to retain these rows to avoid imposing additional complexity and inconvenience on the user.

Additionally, *Check Performance* removed cells and rows that did alter the visual appearance of worksheets. For example, the cover sheet shown in Figure 8a was modified such that the cells responsible for whitespace on the cover sheet were removed, as illustrated in Figure 8b [16].

While this may be acceptable for a cover sheet, it poses a problem for forms such as those shown in Figure 9a, which consist of several empty cells forming a table that might be prepared to be filled out later [19]. The *Check Performance* function in *Excel* removes all these borders because they are outside the rectangle spanned by the values, as shown in Figure 9b.

5 EVALUATION

As part of our evaluation, we processed all *Excel* files [1] obtained from the German Federal Statistical Office (*Destatis*) [1, 2]. The full details of how these files were acquired through a custom web scraping approach are provided in Appendix A. The primary goal of this evaluation is to determine the extent of data reduction achieved through different techniques, including overall reduction, empty cell reduction, whitespace cell reduction, rows outside of value range removal, and column removal. Figure 10 illustrates the distribution of reduction rates (in percentage) for individual worksheets across all categories, broken down by Empty Cells, Whitespace Cells, Rows Outside of Value Range, and Removed Columns. The chart indicates that some worksheets experienced reductions of nearly 100% in file size. Specifically, one worksheet had a reduction rate of over

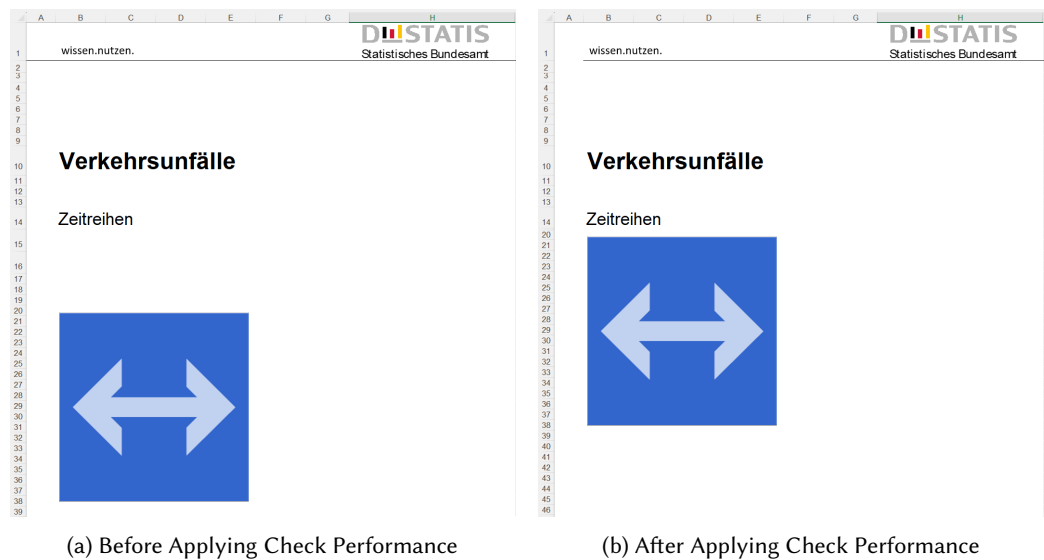


Fig. 8. Comparison of Cover Sheet Before and After Applying Check Performance

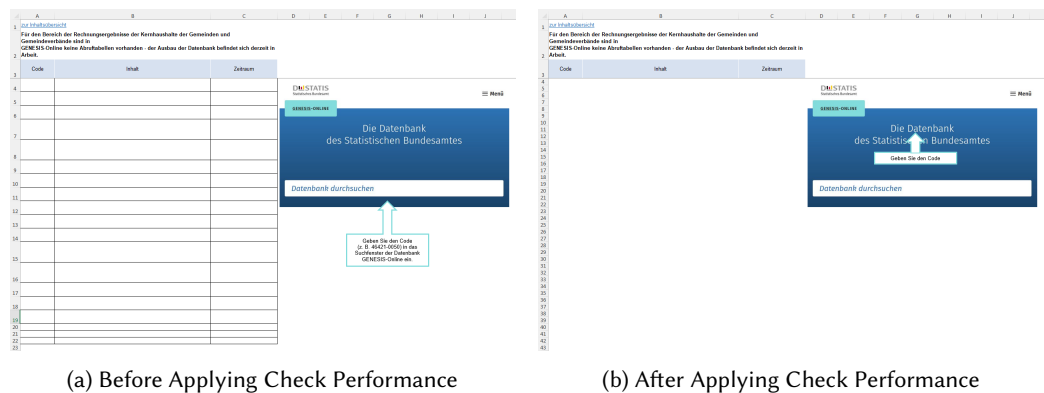


Fig. 9. Deletion of Borders Forming an Empty Table

99.99%; 16 worksheets exceeded 99.9%; and 210 worksheets achieved reductions above 99%. The average reduction rate across all worksheets was 25%.

Most reductions were accomplished by removing rows outside of the value range. The evaluation results focus on the top five files for each reduction factor, as shown in Table 1.

5.1 Overall Reduction Ratio

The top five worksheets with the highest overall reduction ratios are all from the same file, with reduction ratios ranging between 99.95% and 99.99%. This indicates that almost all data in these sheets consisted of file bloat that was successfully eliminated. The repetition of this file in the subsequent table suggests that the high reduction is primarily attributable to a specific reduction category.

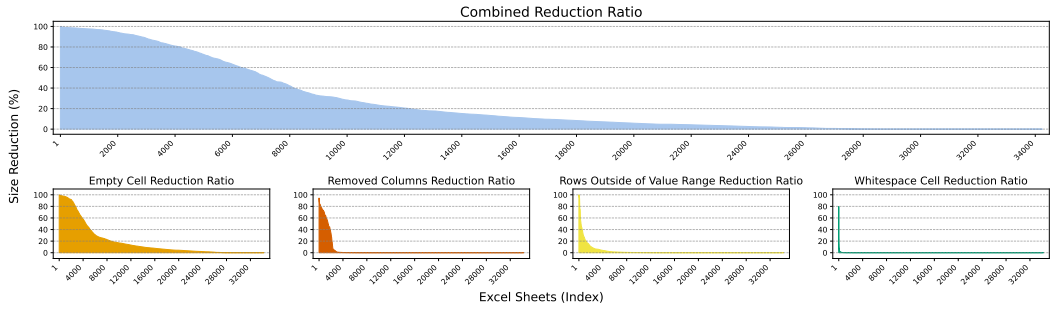


Fig. 10. Distribution of Size Reduction Percentages for Excel Sheets

5.2 Rows Outside of Value Range Reduction Ratio

The majority of unnecessary data consisted of rows outside the value range, with reduction ratios between 99.94% and 99.99% in the top five files. This suggests that the largest portions of redundant data were predominantly from a single category rather than a combination of different categories.

5.3 Empty Cell Reduction Ratio

Empty cell removal achieved significant reductions, with ratios between 99.90% and 99.91% in the top five files. Notably, three different files were affected, although the worksheets shared the same name. This suggests that these explanatory worksheets might have been duplicated from one *Excel* file to another, or the entire file was duplicated, leading to a similar number of empty cells across them.

5.4 Removed Columns Reduction Ratio

Removing unnecessary column definitions resulted in reduction ratios ranging from 92.43% to 94.38% in the top five worksheets. Similar worksheet names were again noticeable, implying that duplicated worksheets carried over a significant amount of file bloat due to unused column definitions.

5.5 Whitespace Cell Reduction Ratio

Whitespace cell removal was the least effective in reducing file size. The top sheet still had a notable 79.71% of the file size attributed to whitespaces, while the second entry dropped to 44.68%, and the last three entries were around one-third. With the exception of the last two files, the top five were distinct. However, the similar worksheet names and high proportion of whitespace suggest that redundant content might have been carried over to other worksheets through duplication.

5.6 Summary of Findings

The evaluation demonstrates that the data reduction techniques employed by *Excel Shrink* are highly effective in eliminating redundant, empty, or irrelevant data across a variety of *Excel* worksheets. Significant reductions—exceeding 90% in several cases—were achieved in sheets containing a high proportion of empty cells, or irrelevant rows and columns. These results highlight the robustness of our reduction methods in processing diverse data formats and structures.

Moreover, the analysis underscores the importance of addressing different reduction factors independently, as their impact on file size varied. While empty cell removal provided substantial

reductions in certain worksheets, others benefited more from the removal of rows or columns. Notably, whitespace removal had a smaller overall impact.

6 EXPERIMENTAL SETUP

To evaluate the behavior of *Excel* regarding how the files are interpreted and saved, as well as to test the behavior of the *Add-in* and the new *Check Performance* feature, we tested with two distinct versions of *Excel* from *Microsoft Office*: *Microsoft Office LTSC Professional Plus 2021* and *Microsoft Excel for Microsoft 365 MSO (Version 2409 Build 16.0.18025.20030) 64-bit*. These versions were selected to evaluate compatibility and performance across different Excel environments.

Our test machine was an *Intel Core i7-4790K* quad-core CPU desktop machine running at 4.00 GHz with 16 GB of RAM. The hardware specifications were chosen to reflect a typical desktop setup for data processing tasks.

Our test machine was an *Intel(R) Core(TM) i7-4790K* quad-core CPU desktop machine running at 4.00GHz with 16GB of RAM.

7 CONCLUSION

We have presented *Excel Shrink*, an open-source tool that significantly reduces the file sizes of *Excel* sheets from *Destatis* by eliminating unnecessary content. Our analysis identified common practices that lead to bloated *Excel* files and demonstrated how targeted removal of redundant data can achieve reductions of up to 99.99% without altering the spreadsheets' appearance.

Our tool outperforms existing solutions in both file size reduction and preservation of formatting. By making *Excel Shrink* available to the community, we hope to facilitate more efficient data processing and analysis of large-scale *Excel* datasets.

REFERENCES

- [1] [n. d.]. Destatis XLSX Files Shrunk with Excel Shrink. https://anonymous.4open.science/r/shrunked_destatis_xlsx_files-4E27/. Accessed: 2024-12-07.
- [2] [n. d.]. Original Destatis XLSX Files Before Shrinking with Excel Shrink. https://anonymous.4open.science/r/original_destatis_xlsx_files-5B7B/. Accessed: 2024-12-07.
- [3] 2024. Excel Shrink UI - An open-source graphical user interface for Excel Shrink. <https://anonymous.4open.science/r/excel-shrink-ui-BE5C/README.md>.
- [4] Rui Abreu, Jácome Cunha, Joao Paulo Fernandes, Pedro Martins, Alexandre Perez, and Joao Saraiva. 2014. Faultysheet detective: When smells meet fault localization. In *2014 IEEE International Conference on Software Maintenance and Evolution*. IEEE, 625–628.
- [5] Mangesh Bendre, Bofan Sun, Ding Zhang, Xinyan Zhou, Kevin ChenChuan Chang, and Aditya Parameswaran. 2015. Dataspread: Unifying databases and spreadsheets. In *Proceedings of the VLDB Endowment International Conference on Very Large Data Bases*, Vol. 8. NIH Public Access, 2000.
- [6] Mangesh Bendre, Tana Wattanawaroon, Kelly Mack, Kevin Chang, and Aditya Parameswaran. 2019. Anti-freeze for large and complex spreadsheets: Asynchronous formula computation. In *Proceedings of the 2019 International Conference on Management of Data*. 1277–1294.
- [7] George Chalhoub and Advait Sarkar. 2022. "It's freedom to put things where my mind wants": Understanding and improving the user experience of structuring data in spreadsheets. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*. 1–24.
- [8] Microsoft Corporation. 2024. Workbook Class (DocumentFormat.OpenXml.Spreadsheet) | Microsoft Learn. <https://learn.microsoft.com/en-us/dotnet/api/documentformat.openxml.spreadsheet.workbook?view=openxml-3.0.1>. Accessed on October 7, 2024.
- [9] Microsoft Corporation. 2024. Working with the shared string table | Microsoft Learn. <https://learn.microsoft.com/en-us/office/open-xml/spreadsheet/working-with-the-shared-string-table?tabs=cs>. Accessed on October 7, 2024.
- [10] Jácome Cunha, Joao Paulo Fernandes, Pedro Martins, Jorge Mendes, and Joao Saraiva. 2012. Smellsheet detective: A tool for detecting bad smells in spreadsheets. In *2012 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. IEEE, 243–244.
- [11] Peter Deutsch. 1996. *DEFLATE compressed data format specification version 1.3*. Technical Report.

- [12] Ecma International. 2021. *ECMA-376: Office Open XML File Formats* (5th ed.). Technical Report ECMA-376. Ecma International. <https://ecma-international.org/publications-and-standards/standards/ecma-376/> ISO/IEC 29500, Technical Committee TC45.
- [13] Kelly Mack, John Lee, Kevin Chang, Karrie Karahalios, and Aditya Parameswaran. 2018. Characterizing scalability issues in spreadsheet software using online forums. In *Extended Abstracts of the 2018 CHI Conference on Human Factors in Computing Systems*. 1–9.
- [14] Microsoft. 2014. *Open XML SDK 2.5 for Office*. Microsoft Press.
- [15] Patrick O’Beirne. 2014. Excel 2013 Spreadsheet Inquire. *arXiv preprint arXiv:1401.7586* (2014).
- [16] Statistisches Bundesamt Destatis (German Federal Statistical Office). 2022. Verkehrsunfälle - Zeitreihen 2021 (Traffic Accidents - Time Series 2021) (Letzte Ausgabe - berichtsweise eingestellt). <https://www.destatis.de/DE/Themen/Gesellschaft-Umwelt/Verkehrsunfaelle/Publikationen/Downloads-Verkehrsunfaelle/verkehrsunfaelle-zeitreihen-pdf-5462403.html>. Accessed on October 7, 2024.
- [17] Statistisches Bundesamt Destatis (German Federal Statistical Office). 2022. Wanderungen - Fachserie 1 Reihe 1.2 - 2021 (Migration - Series 1, Series 1.2 - 2021) (Letzte Ausgabe - berichtsweise eingestellt). <https://www.destatis.de/DE/Themen/Gesellschaft-Umwelt/Bevoelkerung/Wanderungen/Publikationen/Downloads-Wanderungen/wanderungen-2010120217004.html>. Accessed on October 7, 2024.
- [18] Statistisches Bundesamt Destatis (German Federal Statistical Office). 2023. Private Konsumausgaben und Verfügbares Einkommen - 1. Vierteljahr 2023 (Private Consumption Expenditures and Disposable Income - 1st Quarter 2023) (Letzte Ausgabe - berichtsweise eingestellt). <https://www.destatis.de/DE/Themen/Wirtschaft/Volkswirtschaftliche-Gesamtrechnungen-Inlandsprodukt/Publikationen/Downloads-Inlandsprodukt/konsumausgaben-pdf-5811109.html>. Accessed on October 7, 2024.
- [19] Statistisches Bundesamt Destatis (German Federal Statistical Office). 2023. Statistischer Bericht - Rechnungsergebnisse der Kernhaushalte der Gemeinden und Gemeindeverbände 2021 (Statistical Report - Financial Results of the Core Budgets of Municipalities and Municipal Associations 2021). <https://www.destatis.de/DE/Themen/Staat/Oeffentliche-Finanzen/Ausgaben-Einnahmen/Publikationen/Downloads-Ausgaben-und-Einnahmen/statistischer-bericht-rechnungsergebnis-kernhaushalt-gemeinden-2140331217005.html>. Accessed on October 7, 2024.
- [20] Statistisches Bundesamt Destatis (German Federal Statistical Office). 2024. robots.txt. <https://www.destatis.de/robots.txt>. Accessed on October 7, 2024.
- [21] Statistisches Bundesamt Destatis (German Federal Statistical Office). 2024. Statistischer Bericht - Landwirtschaftliche Betriebe - Förderprogramme - 2023 (Statistical Report - Agricultural Enterprises - Funding Programs - 2023). <https://www.destatis.de/DE/Themen/Branchen-Unternehmen/Landwirtschaft-Forstwirtschaft-Fischerei/Landwirtschaftliche-Betriebe/Publikationen/Downloads-Landwirtschaftliche-Betriebe/statistischer-bericht-foerderprogramme-5411206239005.html>. Accessed on October 7, 2024.
- [22] Raymond R Panko. 1998. What we know about spreadsheet errors. *Journal of Organizational and End User Computing (JOEUC)* 10, 2 (1998), 15–21.
- [23] Sajjadur Rahman, Mangesh Bendre, Yuyang Liu, Shichu Zhu, Zhaoyuan Su, Karrie Karahalios, and Aditya G Parameswaran. 2021. NOAH: interactive spreadsheet exploration with dynamic hierarchical overviews. *Proceedings of the VLDB Endowment* 14, 6 (2021), 970–983.
- [24] Sajjadur Rahman, Kelly Mack, Mangesh Bendre, Ruilin Zhang, Karrie Karahalios, and Aditya Parameswaran. 2020. Benchmarking spreadsheet systems. In *Proceedings of the 2020 acm sigmod international conference on management of data*. 1589–1599.
- [25] Prash Shirolkar. 2022. Check Performance in Excel for the Web. <https://techcommunity.microsoft.com/t5/microsoft-365-insider-blog/check-performance-in-excel-for-the-web/ba-p/4216095>. Microsoft 365 Insider Blog, Accessed: 2024-04-27.
- [26] Prash Shirolkar. 2024. *Make Your Workbooks More Performant with Check Performance in Excel for Windows*. <https://techcommunity.microsoft.com/t5/microsoft-365-insider-blog/make-your-workbooks-more-performant-with-check-performance-in/ba-p/4224276> Accessed: 2024-10-18.
- [27] Alaaeddin Swidan, Felienne Hermans, and Ruben Koesoemowidjojo. 2016. Improving the performance of a large scale spreadsheet: a case study. In *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, Vol. 1. IEEE, 673–677.
- [28] Dixin Tang, Fanchao Chen, Christopher De Leon, Tana Wattanawaroon, Jeaseok Yun, Srinivasan Seshadri, and Aditya G Parameswaran. 2023. Efficient and Compact Spreadsheet Formula Graphs. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, 2634–2646.

A SCRAPING DESTATIS

The *Destatis* website does not provide an API for searching publications, making it necessary to interact with the website directly. To automate this process, we developed a web scraper that searches for relevant publications while minimizing the number of requests to the server. The web scraping and downloading process is outlined in Algorithm 1. The URL structure of the *Destatis* website is as follows, with query parameters appended to the base path:

- `/SiteGlobals/Forms/Suche/Expertensuche_Formular.html`:
This is the base path of the URL, which points to the expert search form on the *Destatis* website. It serves as the primary endpoint where the search query is submitted.
- `templateQueryString=2000`:
This query parameter specifies the search year. In this case, the search is focused on publications from the year 2000. By adjusting this value, different years can be searched.
- `gtp=474_list%253D2`:
This parameter handles pagination. It points to a specific page of search results. The value `%253D` is a doubly encoded sequence, where `%253D` is the URL-encoded form of `%3D`, which itself encodes the equal sign (`=`). After decoding twice, the query parameter becomes `gtp=474_list=2`. The number after the second equal sign corresponds to the current page index in the pagination of the search results.
- `documentType_=publication`:
This parameter filters the results by the type of document. By setting this to `publication`, the search is limited to publications, where *Excel* files are usually attached.
- `resultsPerPage=100`:
This parameter determines how many results should be displayed per page. In this case, it is set to `100`, which is the maximum number of results that can be retrieved in a single page request.

Through extensive testing, we discovered that *Excel* files are exclusively found in the “Publication” document type. By using the GET parameter `documentType_=publication`, we were able to significantly narrow down our search results.

Initially, we assumed that searching for the term `xlsx` would identify all publications containing *Excel* file attachments. However, this search only returned about 100 results, far fewer than the actual number of *Excel* files available on the *Destatis* website. Furthermore, there is no option to retrieve a complete list of all publications ever created. To overcome this, we refined our strategy by searching based on publication years. A publication should appear in the search results when queried by its publication year. Although this approach returns some irrelevant results—where the publication year might appear in the text or a file attachment—the number of requests is greatly reduced by caching and reusing links to the *Excel* file attachments. This ensures that the same files are not downloaded multiple times.

The search results on the *Destatis* website, like many others, are paginated, with the interface offering the ability to display 10, 20, or 30 entries per page. The number of results per page can be adjusted using the GET parameter `resultsPerPage=30`. While the interface does not provide an option for more than 30 results per page, we found that manually setting this parameter allowed us to request up to 100 results per page. Through testing, we confirmed that 100 is the maximum number of results that can be returned at once. We set the `resultsPerPage` parameter accordingly to reduce the number of requests further.

Once the results for a page are fetched by the web scraper, we simulate clicking the “next page” link to load subsequent results. The XPath expression `//a[@class='forward button']/@href` is used to identify the link using the CSS class `forward button`.

Algorithm 1 Web Scraping and Downloading Excel Files with Caching

Input: Base URL B, form URL F, download folder D, CSV cache file C, crawl delay (30 seconds)
Initialize: Cached URLs set U, and load file records from CSV C
 Create folder D if it does not exist
 {— Scraping Process —}
for each year in years **do**
 Construct the search URL using year and base parameters
 while there are more pages to fetch **do**
 Send HTTP GET request to search URL
 Parse HTML to extract .xlsx links
 Filter links that are not already in the cache U
 for each new link **do**
 Extract file name from link
 Add new link and file name to file records
 Append new records to cache C
 end for
 Save the updated file records back to CSV C
 Check for the next page button
 if next page exists **then**
 Update URL to the next page
 Wait for CRAWL_DELAY seconds
 else
 Exit the loop
 end if
 end while
end for
 {— Download Process —}
for each new file record **do**
 Download file from the URL
 Save it in the download folder D
 Wait for CRAWL_DELAY seconds
end for

Once we have queried each publication year from 2000 to 2024, and the file attachments have been extracted from the result pages until no further “next page” button is available, the scraping process for the *Excel* files is complete.

A.1 Respecting Crawl Delay

To minimize the load on the *Destatis* website, we adhered to the crawl-delay directive specified in the site’s robots.txt file [20], pausing for 30 seconds between requests. This delay ensures that our scraper operates within the guidelines provided by *Destatis*.

Received 17 October 2024; revised ; accepted

Table 1. Top Sheets by Each Reduction Factor

File	Sheet	Reduction Factor Value (%)
Top 5 rows for Reduction Ratio		
verkehrsunfaelle-zeitreihen-xlsx-5462403.xlsx	Technisch_bedingte_Leerseit	99.99
verkehrsunfaelle-zeitreihen-xlsx-5462403.xlsx	4.6_(3)	99.96
verkehrsunfaelle-zeitreihen-xlsx-5462403.xlsx	3.1_(7)	99.96
verkehrsunfaelle-zeitreihen-xlsx-5462403.xlsx	5.1.4_(2)	99.95
verkehrsunfaelle-zeitreihen-xlsx-5462403.xlsx	3.1_(1)	99.95
Top 5 rows for Rows Outside of Value Range Reduction Ratio		
verkehrsunfaelle-zeitreihen-xlsx-5462403.xlsx	Technisch_bedingte_Leerseit	99.99
verkehrsunfaelle-zeitreihen-xlsx-5462403.xlsx	3.1_(7)	99.96
verkehrsunfaelle-zeitreihen-xlsx-5462403.xlsx	4.6_(3)	99.95
verkehrsunfaelle-zeitreihen-xlsx-5462403.xlsx	3.1_(1)	99.95
verkehrsunfaelle-zeitreihen-xlsx-5462403.xlsx	10.1	99.94
Top 5 rows for Empty Cell Reduction Ratio		
statistischer-bericht-mikrozensus-h...-2010300217005.xlsx	Erläuterung zu CSV-Tabellen	99.91
statistischer-bericht-mikrozensus-h...erstergebnisse.xlsx	Erläuterung zu CSV-Tabellen	99.91
statistischer-bericht-mikrozensus-h...endergebnisse.xlsx	Erläuterung zu CSV-Tabellen	99.91
statistischer-bericht-migrationshin...-2010220227005.xlsx	Erläuterung zu CSV-Tabellen	99.90
statistischer-bericht-migrationshin...-2010220227005.xlsx	Erläuterung zu CSV-Tabellen	99.90
Top 5 rows for Removed Columns Reduction Ratio		
strauchbeerenanbau-2030319227005.xlsx	DE3_2 (4)	94.38
strauchbeerenanbau-2030319227005.xlsx	DE3_2 (3)	93.41
strauchbeerenanbau-2030319227005.xlsx	DE3_2	93.37
strauchbeerenanbau-2030319227005.xlsx	DE3_2 (2)	92.43
fallpauschalen-krankenhaus-2120640167005.xlsx	Erläuterungen Fehlerkorrektur	91.11
Top 5 rows for Whitespace Cell Reduction Ratio		
statistischer-bericht-berufliche-sc...-5211004227005.xlsx	21121-1723	79.71
statistischer-bericht-verkehr-2080110231075.xlsx	csv-46251-28	44.68
ueberschuldung-2150500217005.xlsx	5.1	29.97